Understanding State, Hooks, and `useState` in React

PART 1 — WHAT IS STATE? (FROM REAL LIFE TO PROGRAMMING)

1 What does the word state mean?

The word **state** did not come from programming.
It is a **general concept** that exists everywhere.

State means:

> **The current situation or condition of something at a specific moment in time.**

Examples from daily life:f

- A door can be **open** or **closed**
- A phone can be **on** or **off**
- A person can be **awake** or **asleep**
- A traffic light can be **red**, **yellow**, or **green**

Important observations:

- State always describes **now**
- State can **change**
- What happens next depends on the **current state**

2 Why does the concept of state exist?

Imagine something that **cannot remember**.

Every time you talk to it, it forgets:

- What you said
- What it answered
- What happened before

Such a thing cannot interact meaningfully.

Memory is required for interaction.

Programs are the same.

If a program forgets everything:

- It cannot progress
- It cannot react
- It cannot adapt

That memory is called **state**.

3State in programming

In programming:

> **State is the stored data that represents the current condition of a program while it is running.**

Another way to say it:

> **State is what the program remembers about itself at this moment.**

4 The simplest form of state: variables

```javascript
let score = 0;
```

This line means:

- The program stores a value
- Right now, the score is `0`

That stored value represents the **current state** of `score`.

5 State exists because programs run over time

```javascript
let score = 0;

// later
score = 10;

// later
score = 20;
```

Same variable
Different moments
Different meaning

This is state changing over time

So:

> **State is not just a value — it is a value that can change as the program runs.**

6 When does a variable become “state”?

This is critical.

> **Not every variable is state**

A variable is considered **state** only if:

1. It represents a real situation
2. It can change over time
3. The program behaves differently because of it

Not state (just a calculation)

```javascript
let total = price * quantity;
```

State (controls behavior)

```javascript
let isLoggedIn = false;
```

- Represents a situation
- Can change
- Controls program logic

7 State controls behavior (core idea)

```javascript
let hasKey = false;

function openDoor() {
  if (hasKey) {
    console.log("Door opened");
  } else {
    console.log("Door locked");
  }
}
```

Change the state:

```javascript
hasKey = true;
```

Now the **same code behaves differently**.

State controls behavior.

8 State = memory (repeat on purpose)

This sentence is essential:

> **State is the memory of a program.**

Without state:

- No past
- No progress
- No interaction

PART 2 — FROM PROGRAM STATE TO UI STATE

9 Programs vs web applications

Traditional programs:

- Run
- Finish
- Exit

Web applications:

- Stay alive
- Respond to users
- Change visually

So now:

- State does not only control logic
- State controls **what the user sees**

10 What is UI State?

> **UI State is the data that controls what the interface looks like right now.**

Examples:

- Is a modal open or closed?
- What text is inside an input?
- Which tab is active?
- Is data loading or loaded?

Important:

- These are **not visual elements**
- They are **data** that the UI depends on

11 UI does not decide — state decides

Very important mental shift:

> **The UI is a reflection of state.**

- Change state → UI changes
- UI does not manage itself
- Data controls visuals

PART 3 — REACT COMPONENTS

12 What is a React component?

A React component is:

> **A JavaScript function that describes what the UI should look like based on data.**

It does not:

- Directly change the DOM
- Decide behavior on its own

It **describes** UI.

13 The core React formula

This formula explains React completely:

> **UI = f(data)**

In React:

- `data` = **props + state**

So:

> **UI = f(props, state)**

14 Components are functions (and this is a problem)

JavaScript functions **do not remember values**.

```javascript
function example() {
  let x = 0;
  x++;
  return x;
}
```

Every call returns `1`.

Why?

- Function variables are destroyed after execution
- Functions start fresh every time

Functions have no memory.

15 But components need memory

React components must:

- Remember state
- Persist values
- React to changes

But they are just functions.

This is the core problem React had to solve.

PART 4 — WHY HOOKS EXIST

16 Why normal variables don’t work in React

```JavaScript
function Counter() {
  let count = 0;
  count++;
}
```

On every render:

- `count` resets
- State is lost
- UI becomes inconsistent

This breaks React’s promise.

17 React’s solution: Hooks

React introduced **Hooks** to solve this exact problem.

> **Hooks allow function components to connect to React’s internal systems.**

18 What is a Hook? (deep definition)

> **A Hook is a special function that lets a component “hook into” React’s features.**

Hooks give access to:

- State
- Lifecycle
- Context
- Side effects

Hooks are **not normal helper functions**.
They are a **contract with React**.

19 Why Hooks start with `use`

The prefix `use` means:

- "I want to use a React capability"

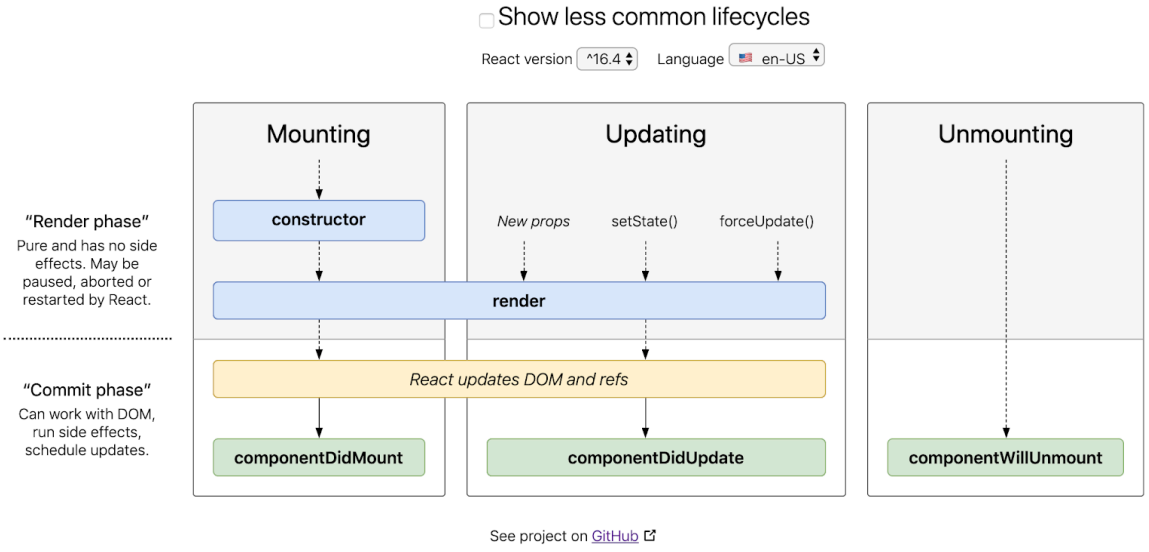
Examples:

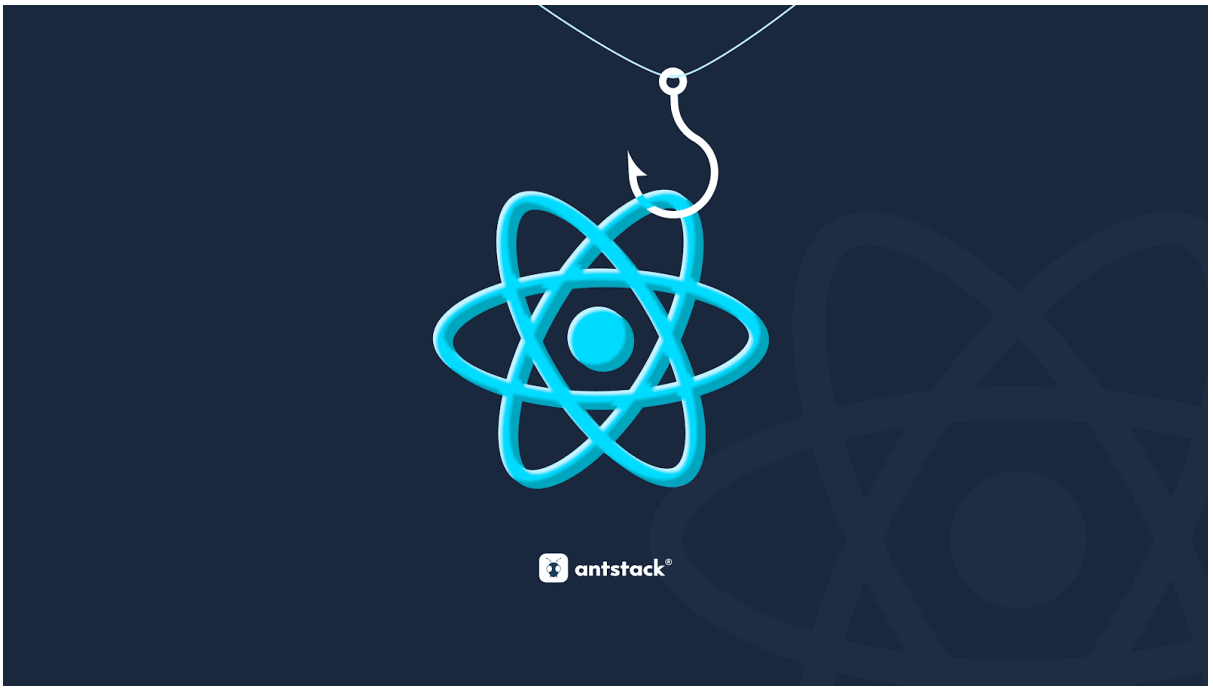
- `useState` → use React's state system
- `useEffect` → use React's effect system

This allows React to:

- Track hooks
- Enforce rules
- Keep state stable

20 Hooks as a bridge (mental model)





Think of it as:

- Component → plain function
- React → engine
- Hooks → connection cable

Without hooks, components have **no memory**.

PART 5 — `useState` AS THE SOLUTION

21 What problem does `useState` solve?

We know:

1. State is memory
2. Functions forget
3. Components need memory

So React gives us:

> **`useState` — the Hook that gives memory to function components**

22 What happens when `useState` is used?

When a component uses `useState`:

1. React creates a memory cell
2. React keeps it between renders
3. React watches it for changes
4. When it changes → React re-renders the component

23 Understanding `useState` conceptually

`useState` provides:

- The **current state value**
- A **safe way to update it**

You never change state directly.
You **ask React** to update it.

24 Why state updates must go through React

React needs to:

- Schedule rendering
- Batch updates
- Optimize DOM changes

So state updates are **requests**, not assignments.

25 `useState` syntax (now it makes sense)

```
None
const [count, setCount] = useState(0);
```

Explanation:

- `count` → current situation
- `setCount` → request React to change the situation
- `0` → initial state

26 The most important rule in React

Wrong:

```
JavaScript
count++;
```

Correct:

```
JavaScript
setCount(count + 1);
```

Because:

- React must be notified
- React controls rendering

27 What happens when state changes?

1. State changes
2. React detects the change
3. Component function runs again
4. New UI is calculated
5. DOM updates automatically

28 The key mental model

A React component is a function that re-runs whenever its state changes.